

What experiments 5, 7 and 9 do

A reading of `exp5_inactivity.jl`, `exp7_convergence_rate.jl` and `exp9_second_order.jl`

Read against branch `checking-w-Claude-Code`, 2026-08-20

All three files are read as source and cross-checked against their most recent archived run (2026-08-19, 16:28). Where the code and the archive disagree, the disagreement is reported rather than reconciled.

Summary in one paragraph each

Experiment 5 measures whether the trust-region constraint ever *stops* binding. The claim is that the mechanisms split cleanly into two groups by this property, and – the part that matters – that the split is *invisible* to a first-order convergence test, because both groups drive $\|g_k\| \rightarrow 0$ perfectly well. The observable is the fraction of late iterations with $\|s_k\| = \Delta_k$. Two configurations are included with parameters deliberately set below any plausible threshold, as a positive control: they should stay pinned at 1 while every other rule falls to 0.

Experiment 7 measures the *observed local convergence order* of each radius mechanism and tests a negative claim: that the order does *not* depend on the mechanism. Once the trust-region constraint stops binding, the step is the unconstrained model minimiser, so the rate is a property of the model rather than of the radius rule. What the mechanism controls is *when* that happens. The script therefore fits a rate only on the inactive tail of each run, and reports the fitted order beside k_* , the first permanently inactive iteration.

Experiment 9 tests what happens when a radius rule anchored to $\|g\|$ meets a saddle point. At a saddle $\|g\| = 0$ exactly as at a minimiser, so a $\|g\|$ -anchored rule sets the radius it would use at a solution and the run stops, reporting success. The script demonstrates the collapse, then shows that repairing it needs *three* ingredients simultaneously: the criticality measure $\tau = \max\{\|g\|, -\lambda_{\min}\}$, a subsolver that can move along a negative-curvature direction, and a model capable of reporting negative curvature at all.

1 Experiment 5: direct measurement of inactivity

1.1 The claim under test

Part II divides the mechanisms by whether the trust-region constraint eventually stops binding. Experiment 5 measures that division directly, and its real subject is the second half of the claim:

Both groups drive $\|g_k\| \rightarrow 0$ monotonically. A first-order convergence test cannot tell them apart.

This is why the experiment exists as a separate script rather than as a column in the comparison table. A method whose constraint binds for ever still converges, and still reports `:first_order`. What it loses is the local rate: the step is never the unconstrained model minimiser, so the superlinear phase never arrives. Nothing in $\|g_k\|$, in ρ_k , or in the exit status reveals this. Only the activity flag does.

1.2 The observable, and the floor placed under it

The observable is `:active_trajectory`, the per-iteration Boolean $\|s_k\| = \Delta_k$ that the solver records when `trace = true`. It is reduced to a single number per run by `active_fraction`, the share of the last 10% of iterations on which the constraint bound.

That reduction is fragile on short runs, and the file guards it:

```
const MIN_ITERS_FOR_TAIL = 30

tail_active(r::RunRecord; frac::Float64 = 0.1) =
    length(r.active_traj) < MIN_ITERS_FOR_TAIL ? NaN :
        active_fraction(RecordView(r); tail = frac)
```

The reasoning is in the docstring: on a ten-iteration run the last 10% is a single iteration, so `tail_active` can only be 0 or 1 and a table of such values measures rounding rather than asymptotics. Runs below the floor return `NaN`, are excluded from the mean, and are counted separately in a `measured` column. Section 1.8 argues that this guard, while correct, has a consequence the summary table does not currently account for.

1.3 The rule list, and two deliberate controls

Experiment 5 does not use the shared `RULES`. It defines its own six:

configuration	why it is in the list
<code>RDelta</code>	Classical, criticality-blind. Expected to go inactive.
<code>RStep</code>	Step-driven. Expected to go inactive.
<code>RDF0(ζ=1)</code>	Criticality-anchored at a sane parameter.
<code>RDF0(ζ=0.01)</code>	Control. ζ far below any plausible $\bar{\kappa}$.
<code>RGrad</code>	Gradient-scaled, uncapped.
<code>RGradCapped(μ = μ_{max} = 0.05)</code>	Control. The cap pins the multiplier below threshold, so the region can never grow enough to release.

The two controls are the design’s strongest feature. The closing note says why explicitly: their parameters sit below any plausible $\bar{\kappa}$ ($4/\lambda_{\min}^*$ under `:eigenvalue`, $8/\lambda_{\min}^*$ under `:neighbourhood`), so they *should* stay pinned near 1 while every other rule falls to 0. An experiment that only shows the expected group succeeding proves much less than one carrying a case engineered to fail in a known way. If a control came out inactive, the measurement would be wrong.

1.4 What the script does

1. Opens an archive tagged `inactivity` and records the six rules, the shared `SOLVER_PARAMS` and the problem selection.
2. Runs the (problem $\times$ rule) grid through `run_experiment` with `trace = true` and, crucially, with `solver_kwargs = (hessian_norm = true,)`.
3. Reduces each run to a tail-active fraction and aggregates per rule.
4. Emits the three radius series *per run*, without averaging.
5. Draws a bar chart of mean tail activity and a cumulative-activity plot.

The `hessian_norm = true` in step 2 is not incidental. It makes the solver record $\|H_k\|$ at every iterate, which is what lets `radius_sums` form

$$\sum_k \frac{\Delta_k^2}{M_k}, \quad M_k = L + \max_{i \leq k} \|H_i\|,$$

and the in-line comment states the reason: *that*, not $\sum_k \Delta_k^2$, is what Part II proves finite for the criticality-anchored rules. The two agree only while $\{\|H_k\|\}$ is bounded, and that boundedness

is precisely the hypothesis the M_k form was introduced to drop. Experiment 5 is currently the only script in the suite that populates this column.

1.5 Two refusals to average

Both are deliberate and both are worth noting, because each removes a way the table could mislead.

No solved filter. The comment is blunt: a configuration trapped below its threshold is the one that exhausts the budget, so filtering on success would remove exactly the evidence the experiment exists to collect.

No averaging of the radius series across problems. The second table prints one row per run. Averaging $\sum_k \Delta_k$ across problems would mix a divergent series with convergent ones and produce a number with no reading. The reader is told instead to read the shape across rules *on the same problem*.

1.6 Outputs

artefact	content
exp5_inactivity_summary.txt	Per rule: runs, solved, measured, mean tail activity, never inactive. The table the claim lives in.
exp5_radius_series.txt	Per run: iterations, status, $\sum \Delta_k$, $\sum \Delta_k^2$, $\sum \Delta_k^2/M_k$.
exp5_tail_active.pdf	Bar chart of mean tail activity with a dashed line at 0.5.
exp5_active_trajectory_AKIVA.pdf	Cumulative active fraction along the run. Named for the first problem in the set, and drawn for that one only.

1.7 What the archived run shows

From exp_2026-08-19_16-28-10_inactivity:

rule	runs	solved	measured	mean tail act	never inactive
RDelta	3	3	1	0.000	0
RStep	3	3	1	0.800	0
RDF0($\zeta=1$)	3	3	1	0.000	0
RDF0($\zeta=0.01$)	3	3	2	0.600	1
RGrad	3	3	1	0.400	0
RGradCapped	3	3	3	1.000	3

The headline claim is confirmed, and cleanly. Every configuration solves every problem – the solved column is 3/3 throughout – while RGradCapped never goes inactive on any of them. A first-order test sees six identical successes; the activity column sees two distinct behaviours. That is the experiment’s whole point, and it lands.

The controls behave as designed. RGradCapped($\mu = 0.05$) sits at mean tail activity 1.000 with 3/3 never inactive. RDF0($\zeta = 0.01$) is partly trapped at 0.600 with 1/3 never inactive. The cost of being trapped is visible in the second table: on BEALE, RDF0($\zeta=0.01$) takes **2835** iterations against RDF0($\zeta=1$)’s **7**, and RGradCapped takes 566. Same problem, same tolerance, same first-order status.

The $\sum \Delta_k^2/M_k$ column is populated and spans eight orders of magnitude, from 6.6×10^{-5} (RStep on AKIVA) to 7.4×10^3 (RGrad on BOXBODLS), which is the shape the section is meant to display.

1.8 The selection effect in the `measured` column

Four of the six rules have exactly one measured run. The mean tail activity for `RDelta`, `RStep`, `RDF0( $\zeta=1$ )` and `RGrad` is computed from a single number, because their other two runs finished in fewer than `MIN_ITERS_FOR_TAIL` = 30 iterations and were excluded.

The exclusion is individually correct – a one-iteration tail carries no information – but the runs it removes are not a random sample. Run length is *anti-correlated* with the property being measured: a rule that goes inactive converges quickly and falls below the floor, while a trapped rule grinds on and is always measured. So the untrapped group is assessed on its slowest, least representative runs, and the trapped group on all of its.

Two consequences, and they pull in opposite directions.

- **The bias is conservative for the headline claim.** A slow untrapped run is the one most likely to still be binding late, so measuring only those should *inflate* the untrapped group’s tail activity. Observing `RDelta` and `RDF0( $\zeta=1$ )` at 0.000 anyway is therefore stronger evidence than it looks, not weaker.
- **It is not conservative for the intermediate values.** `RStep` at 0.800 and `RGrad` at 0.400 are each one run. `active_fraction`’s own docstring notes that Part II predicts no intermediate value in the limit, so an intermediate value is a finding – but on a single run of a few dozen iterations the likelier reading is simply that the limit has not been reached. As printed, the table does not distinguish those two readings.

The `measured` column is what makes the effect visible at all, so the instrumentation is doing its job. What is missing is that the mean is presented beside it without qualification. Reporting the count alongside each mean in the manuscript, or raising the problem set until the floor stops binding, would settle it. As with experiment 7, the archived run used only three two-variable problems (AKIVA, BEALE, BOXBODLS), which is the root cause here too: on a larger problem set most runs would clear 30 iterations honestly.

2 Experiment 7: observed local convergence order

2.1 The claim under test

Part II’s inactivity theorem says that beyond some iteration $k_\star$ the constraint $\|s_k\| \leq \Delta_k$ never binds again. Beyond $k_\star$ the step solves the unconstrained model problem, so the iteration is the underlying (quasi-)Newton iteration and its rate is whatever that iteration gives. The radius rule has dropped out.

The experiment turns this into two measurable columns:

- **median order** should be roughly *equal* across mechanisms;
- **median $k_\star$** should *differ* substantially, because that is the quantity the mechanism decides.

A single table showing both is the whole experiment. Note that the claim is a negative one, which is why the design matters more than usual: an artefact of estimation would show up as a spurious difference in the first column.

2.2 Why the estimate must be conditioned on inactivity

This is the methodological core of the file, and the header says so explicitly. The order is estimated as the least-squares slope of $\log\|g_{k+1}\|$ on $\log\|g_k\|$: slope 1 is linear convergence, slope 2 quadratic.

If the fit includes iterations where the constraint was *active*, it mixes a linear phase (radius-limited steps) with the asymptotic phase (full Newton steps) and returns a slope between the

two. That intermediate number looks like a convergence order and is not one. It is an artefact of pooling two regimes.

So the script trims every trajectory to its inactive tail before fitting. The helper is small and worth reading in full:

```
function inactive_tail(r::RunRecord)
    (isempty(r.active_traj) || isempty(r.grad_traj)) && return Float64[]
    k = length(r.active_traj)
    while k >= 1 && !r.active_traj[k]
        k -= 1
    end
    start = k + 1
    start > length(r.active_traj) && return Float64[]
    lo = min(start, length(r.grad_traj))
    return r.grad_traj[lo:end]
end
```

It walks backwards from the end of the activity flags over the run of trailing `false` entries and returns the gradient trajectory from the first of them. If the constraint was still binding on the final iteration, the loop exits at `k == length(...)` and the function returns an empty vector, which the caller treats as the finding for that configuration rather than as missing data.

2.3 What the script does, in order

1. Opens an archive tagged `convergence_rate` and writes the config, recording the rule list, solver parameters and problem selection.
2. Builds the problem set from `default_problems()` (CUTEst when available, the analytic set otherwise) and the configuration list from `rule_configs()`, which pairs every rule in `RULES` with the default model (`ExactHessian`) and default subsolver (`SteihaugCG`).
3. **Overrides the tolerance to 10^{-10} .** This is deliberate and is the one parameter the experiment does not inherit: the asymptotic regime is precisely what is being measured, so the run must be allowed deep into it. The other four parameters are taken from `SOLVER_PARAMS` unchanged.
4. Runs everything through `run_experiment` with `trace = true`, so each result is a `RunRecord` carrying the full trajectories.
5. For each record, computes three quantities and prints a row.
6. Aggregates per rule into a second table, and draws two figures.

2.4 The three quantities per run

quantity	meaning
$ki = k_*$	<code>inactivity_index(RecordView(r))</code> : the first iteration after which the constraint <i>never</i> binds again. nothing if it bound to the end. This is the quantity the mechanism controls.
<code>fi</code>	<code>findfirst(!, r.active_traj)</code> : the first inactive iteration <i>of any kind</i> . Printed as <code>fi - 1</code> to count from zero. It differs from k_* whenever the region releases and then binds again, and printing both makes that visible.

quantity	meaning
ord	<code>estimate_convergence_order(tail)</code> , the least-squares slope described above, computed only when the tail has at least <code>MIN_TAIL = 6</code> points.

Each run is then classified into exactly one of three buckets, and this classification is the part that keeps the summary honest:

- **never** – the constraint bound to the end, so there is no asymptotic phase to fit;
- **short** – fewer than six inactive iterations, too few for a regression that means anything;
- **fitted** – contributes its order and its k_* to the medians.

Only the third bucket enters the medians, and the counts of the other two are printed beside them. The file states the reason directly: a median taken over the runs that reached inactivity early is biased towards the mechanisms that get there early. Without those two counts the summary would flatter exactly the rules the experiment is trying to distinguish.

There is also a deliberate omission. The loop applies **no solved** filter. A run that never goes inactive is not dropped; it is counted in `n_never` and reported. As the comment puts it, that is the result for the configuration rather than a gap in the table.

2.5 Outputs

artefact	content
<code>exp7_conv_order_summary.txt</code>	One row per (problem, rule): status, iterations, k_* , first inactive, tail length, fitted order.
<code>exp7_order_by_rule.txt</code>	One row per rule: how many runs were fitted, short and never; median order; median k_* . This is the table the claim lives in.
<code>exp7_conv_order_boxplot.pdf</code>	Order per rule as a boxplot, with dashed lines at 1 and 2 so linear and quadratic are readable at a glance.
<code>exp7_conv_order_scatter.pdf</code>	Order against k_* . If the claim holds this is a horizontal band: the vertical position (rate) is common, the horizontal spread (when inactivity arrives) is not.

2.6 What the archived run actually shows

From `exp_2026-08-19_16-28-37_convergence_rate`:

rule	fitted	short	never	median	order	median k_*
RDelta	3	0	0	1.462		2
RStep	1	2	0	1.445		1
RDF0	3	0	0	1.444		1
RGrad	1	2	0	1.415		0
RGradCapped	1	2	0	1.415		0
RAdaptiveStep	1	2	0	1.445		1
RAdaptiveGrad	3	0	0	1.611		6

Read as intended: the median order sits in a narrow band, 1.415 to 1.611, while median k_* ranges from 0 to 6. That is the predicted pattern – common rate, different onset.

But the table must not be reported as it stands. Three observations, in increasing order of seriousness.

1. **The problem set is three problems** – AKIVA, BEALE, BOXBODLS – because the archived config records `min_var = 2` and `max_var = 2`. `config.jl` anticipates exactly this and forbids it in a comment on `PROBLEM_LIMIT`: “*Three two-variable problems cannot resolve an asymptotic claim ... Drop to 3 deliberately when debugging a script, and never report a table produced at that setting.*” The committed default is `MAX_VAR = 200`; the archive was produced with it narrowed to 2.
2. **Four of the seven rules have a single fitted run.** A “median” over one observation is that observation. The `short` column shows it plainly (2 of 3 runs discarded for four rules), which is the column doing its job, but it means the narrow band in column four is not yet evidence.
3. **The archived config misstates the tolerance.** `save_config` records `SOLVER_PARAMS`, whose `tol` is 10^{-5} , but the script constructs and uses a local `TRParams` with `tol = 1e-10`. The archive therefore documents a run that did not happen. This matters more here than elsewhere, because the tolerance override is the one thing that makes the asymptotic measurement possible.

3 Experiment 9: first- versus second-order anchoring

3.1 The four claims

The header states four, and the script is organised around them.

-
- | | |
|-----|---|
| (a) | The anchor collapse. Started so the trajectory runs into a saddle, a $\ g\ $ -anchored rule reports the radius it would use at a solution. <code>RGrad</code> sets $\Delta = \mu\ g\ \rightarrow 0$ and halts. <code>RDF0</code> finds $\Delta > \zeta\ g\ = 0$ at every iteration and contracts geometrically. Under $\tau = \max\{\ g\ , -\lambda_{\min}\}$ both hold the radius at the scale of the available curvature and escape. |
| (b) | The measure gap. The ratio $\tau_k/\ g_k\ $ is 1 wherever the model is convex and blows up near a saddle, which is exactly where a first-order diagnostic reports success. |
| (c) | The subsolver is half the story. τ keeps the radius positive; it does not make the step go anywhere. |
| (d) | The model is the other half. Over a positive semidefinite model $\lambda_{\min} \geq 0$ identically, so $\tau \equiv \ g\ $ and the apparatus is an expensive no-op that will still report <code>:second_order</code> at a saddle. |
-

3.2 The test problems

The primary problem is chosen so the failure needs no tuning to provoke:

$$f(x, y) = \frac{1}{4}x^4 - \frac{1}{2}x^2 + \frac{1}{2}y^2, \quad x_0 = (0, 0.7).$$

There is a saddle at the origin with $\lambda_{\min} = -1$ and minimisers at $(\pm 1, 0)$ with $f = -0.25$. The decisive property is that $\partial f / \partial x = x^3 - x$ vanishes when $x = 0$, so the x -axis is invariant under the gradient flow: a start on the y -axis runs *exactly* into the saddle. No parameter has to be tuned to make the pathology appear, which is what makes it a fair test rather than a constructed one.

A second problem, the monkey saddle $x^3 - 3xy^2 + z^2/2$ from $(0.05, 0, 0.6)$, guards against the finding being an artefact of one hand-built quartic. Its origin is a degenerate critical point with a cubic escape direction.

The helper `limit_point(x)` classifies where a run stopped into "saddle", "minimiser" or "other" by distance, which is what turns a trajectory into a table entry.

3.3 Parts (a) and (c): the measure $\times$ subsolver grid

`CONFIGS` is a $2 \times 2 \times 2$ grid: two rule families (`RGrad`, `RDF0`) $\times$ two measures ($\|g\|$, τ) $\times$ two subsolvers (`SteihaugCG`, `EigenPoint(SteihaugCG())`). Each entry also carries the `tol_H` it should run with: -1 disables the second-order stopping test for the $\|g\|$ rows, and 10^{-6} enables it for the τ rows. That coupling is correct – a second-order stopping test over a first-order measure would be incoherent – but it does mean measure and stopping test vary together, so the grid isolates the *pair*, not the measure alone.

`anchor_grid()` runs all eight through `tr_solve` directly rather than through the harness. The file explains why: each is a statement about a single trajectory on one hand-built problem, and routing it through the record layer would add machinery without adding information.

3.4 Part (b): the measure gap

Three figures carry it, and the separation between them is deliberate:

- `plot_radius` – Δ_k per configuration on a log scale: the collapse under $\|g\|$, the plateau under τ .
- `plot_measure_gap` – τ_k and $\|g_k\|$ on the same axes.
- `plot_gap_ratio` – the ratio $\tau_k/\|g_k\|$ on its own axis, with a dotted line at 1. The docstring justifies the duplication: the ratio is the quantity with the interpretation, since where it exceeds 1 the gradient alone calls the point critical and the curvature does not.
- `plot_curvature` – $\lambda_{\min}(B_k)$, negative exactly where the first-order measure misleads.

3.5 Part (d): the blind spot

`blind_table()` holds the rule fixed at `RGradTau` and the subsolver at `EigenPoint`, and varies only the model over `ExactHessian`, `SR1Model` and `LBFSGModel`. Because L-BFGS enforces $B \succ 0$ by construction, $\lambda_{\min} > 0$ always, so $\tau \equiv \|g\|$ and the second-order variant is identical to its first-order twin – while still able to report `:second_order`, because that status is a claim about B and not about $\nabla^2 f$. The file calls this the cheapest demonstration in the suite that a status is only as strong as the model behind it.

3.6 Part (e): the paired sweep

`paired_sweep` runs every $\|g\|$ -anchored rule immediately beside its τ -twin through the shared `run_experiment`, so these runs are archived and resumable like every other experiment. The table to read is `success_table(...; second_order = true)`, whose last column counts *certified* `:second_order` stops. The reading rule is stated crisply: a configuration solving every problem while certifying none has been running a first-order method under a second-order name.

Two harness changes were needed for this part, and the file records both because each would have inverted the conclusion:

- `solved` now counts `:second_order`. It previously tested `== :first_order`, so a τ -run that certified a minimiser scored as a *failure*. The columns doing best would have shown zero reliability.
- `RunRecord` now carries `tau_traj` and `lambda_traj`. They were dropped, so claim (b) could not be plotted from an archived run at all.

3.7 What the archived run actually shows

From `exp_2026-08-19_16-28-45_second_order`:

configuration		status	iters	limit	f(x*)	Delta_end
RGrad	g Steihaug	first_order	1	saddle	0.000000	0.000e+00
RGrad	g EigenPoint	first_order	1	saddle	0.000000	0.000e+00
RGrad	tau Steihaug	stalled	2	saddle	0.000000	2.000e+00
RGrad	tau EigenPoint	second_order	3	minimiser	-0.250000	0.000e+00
RDF0	g Steihaug	first_order	1	saddle	0.000000	5.000e-01
RDF0	g EigenPoint	first_order	3	minimiser	-0.250000	5.000e-01
RDF0	tau Steihaug	stalled	2	saddle	0.000000	2.000e+00
RDF0	tau EigenPoint	second_order	2	minimiser	-0.250000	1.000e+00

Claim (a) is confirmed sharply: both $\|g\|$ -anchored RGrad rows stop after *one* iteration at the saddle with $\Delta_{\text{end}} = 0$, reporting `:first_order`. The table’s own footnote makes the right point – the run has not failed by its own test, since $\|g\|$ really is zero there. It is the test that is wrong.

Both $\tau + \text{Steihaug}$ rows end `:stalled` at the saddle with the radius held at 2.0: τ kept the radius alive, and Steihaug had no reliable negative-curvature step to spend it on. That is claim (c)’s first half, cleanly.

Claim (c) is contradicted by one row. The header says only τ and EigenPoint together reach a minimiser. But RDF0 |g| EigenPoint reaches the minimiser at $f = -0.25$ with the first-order measure. For RDF0, EigenPoint alone was sufficient on this problem. The claim holds for RGrad – whose radius collapses to exactly zero, leaving nothing for any subsolver to work with – but not as a general statement about both families. Either the claim should be narrowed to RGrad, or the difference between the two families should be explained.

3.8 The blind-spot table is sharper than advertised

model	status	iters	limit	min lambda_min	certified?
ExactHessian	second_order	3	minimiser	-1.0000	yes
SR1Model	second_order	1	saddle	1.0000	yes
LBFGSModel	second_order	1	saddle	1.0000	yes

The intended finding is the LBFGSModel row: a certified `:second_order` status at the saddle, from a model structurally incapable of seeing negative curvature.

But SR1Model behaves identically here, and `config.jl` lists SR1 among the models that *can* report $\lambda_{\min} < 0$. The reason is visible in the iteration count: the run stops after one iteration, and an SR1 model initialised at the identity has not yet absorbed a single secant pair, so it reports $\lambda_{\min} = 1$. The blind spot is therefore not only structural but also transient – a model that could see the curvature has not yet had the chance. That is arguably a stronger result than the one claimed, and it is currently unremarked.

3.9 The paired sweep is thin

configuration	solved	median	mean	tail	active	2nd order
RGrad	1/2	20.0	20.0		1.000	1/2
RGradTau	1/2	3.0	3.0		1.000	1/2
RGradCapped	0/2	--	--		--	0/2
RGradCappedTau	1/2	3.0	3.0		1.000	1/2
RDF0	1/2	2.0	2.0		0.500	1/2
RDF0Tau	1/2	2.0	2.0		0.500	1/2
RAdaptiveGrad	0/2	--	--		--	0/2
RAdaptiveGradTau	1/2	2.0	2.0		0.500	1/2

With only two problems every cell is 0/2, 1/2 or 2/2, and no configuration exceeds 1/2. The within-pair comparison is still legible – `RGrad` takes 20 iterations against `RGradTau`’s 3, and `RGradCapped` solves nothing while its τ -twin solves one – but the counts cannot support a reliability statement. More saddle problems are needed before this table carries weight in the manuscript.

4 Shared machinery both files rely on

component	role
<code>RunRecord</code>	One archived run: status, counts, and every trajectory, including <code>tau_traj</code> and <code>lambda_traj</code> for second-order runs.
<code>RecordView</code>	Dresses a <code>RunRecord</code> as a solver result so the package’s diagnostics (<code>inactivity_index</code> , <code>active_fraction</code> , <code>observed_order</code> , ...) apply unchanged to an archived run. Its docstring gives the reason: without it every experiment re-implements the same tail arithmetic, which is how a run and a table come to disagree.
<code>run_experiment</code>	Runs the (problem $\times$ config) grid, archiving and skipping work already present so a campaign can resume.
<code>solved</code>	<code>:first_order</code> or <code>:second_order</code> .
<code>active_fraction</code>	Share of the last tail of a run on which the constraint bound. Used by experiment 5 through <code>RecordView</code> . Its docstring records the prediction that no intermediate value survives in the limit, so one is a finding.
<code>inactivity_index</code>	k_* , the first permanently inactive iteration, or <code>nothing</code> . Used by experiments 5 and 7.
<code>radius_sums</code>	$\sum \Delta_k$, $\sum \Delta_k^2$ and $\sum \Delta_k^2/M_k$, the last being NaN unless the run was traced with <code>hessian_norm = true</code> .
<code>ExperimentArchive</code>	Timestamped directory with <code>experiment_config.toml</code> , <code>tables/</code> , <code>figures/</code> and <code>data/</code> , closed by <code>finalize_archive</code> .

A note on the two order estimators

There are two, and they are not interchangeable:

- `estimate_convergence_order(grad_traj; tail)` in `benchmark/harness.jl` takes a *bare vector* and assumes the caller has already trimmed it. Its docstring warns about this in a boxed note. Experiment 7 uses this one, and does the trimming itself.
- `observed_order(stats; after_inactivity, last, use)` in `src/Diagnostics/diagnostics.jl` takes a full result and does the trimming internally, and additionally skips rejected steps.

Experiment 7’s hand-rolled `inactive_tail` predates `RecordView` being available to it, and duplicates logic `observed_order` already has. The two also differ in a detail that could move a reported number: `observed_order` filters out rejected steps before fitting, and `estimate_convergence_order` does not. Consolidating on the former, called through a `RecordView`, would remove the duplication and the discrepancy at once. Experiment 5 already takes that route for `active_fraction` and `radius_sums`.

5 Findings, collected

	observation	consequence
1	exp5 and exp7 both ran on 3 two-variable problems	<code>config.jl</code> explicitly forbids reporting a table at that setting. Re-run at <code>MAX_VAR = 200</code> before either set of numbers is used. This is also the root cause of finding 8.
2	<code>exp7</code> records <code>tol = 1e-5</code> in the archive but runs at <code>1e-10</code>	The archive documents a run that did not happen. <code>save_config</code> should receive the local <code>params</code> , not <code>SOLVER_PARAMS</code> .
3	<code>exp9</code> claim (c) contradicted by the <code>RDF0 g EigenPoint</code> row	That row reaches the minimiser without τ . Narrow the claim to <code>RGrad</code> or explain the difference between the families.
4	<code>exp9</code> 's <code>SR1</code> row is blind for a different reason than <code>L-BFGS</code>	Structural blindness versus a model that has not yet learned. Worth stating; it strengthens the section.
5	<code>exp9</code> 's paired sweep has two problems	Every cell is 0/2 or 1/2. Add saddle problems before quoting reliability.
6	All three files end with a <code>PROGRAM_FILE</code> guard	None can run standalone: <code>RULES</code> , <code>SOLVER_PARAMS</code> and the harness come from <code>initialisation.jl</code> . The guard, and the usage line in each header, promise an entry point that does not work. This is the same defect that kept experiment 8 out of the suite.
7	<code>exp7</code> duplicates <code>observed_order</code> 's trimming	And differs from it by not skipping rejected steps.
8	<code>exp5</code> 's <code>measured</code> column hides a selection effect	Run length is anti-correlated with the property measured, so four of six rules are scored on a single, atypically slow run. The bias is conservative for the headline claim but not for the intermediate values (<code>RStep</code> 0.800, <code>RGrad</code> 0.400). Report n beside each mean, or enlarge the problem set.
9	<code>exp5</code> is the only script populating $\sum \Delta_k^2 / M_k$	It passes <code>solver_kwargs = (hessian_norm = true,)</code> . This is the column <code>tab: sums</code> in the manuscript leaves provisional on every row, so the machinery to fill it already exists and is already exercised here.

Findings 1, 2 and 6 are defects in the instrumentation rather than in the science. Findings 3 and 4 concern what the manuscript may claim from experiment 9, and 5 and 8 concern how much weight the corresponding tables can carry. Finding 9 is an opportunity rather than a defect.